

Web Accessibility

Laboratory of
Typfhloinformatics

Agenda

1. Introduction to Web Accessibility
2. WCAG 2.1. - Basic
3. Use of assistive technology
4. Semantic code
5. Accessible Rich Internet Applications "WAI-ARIA"

Introduction to Web Accessibility



What is Web Accessibility?

The inclusive practice of removing barriers that prevent interaction with, or access to, websites by people with disabilities.

„**a11y**” is Twitter shorthand for accessibility - there are 11 letters between the a and y.

Use **#a11y** on social media to find the latest articles and discussions.

Accessible technology for all

**„The goal is universal design that is integrated and equal:
Don't make disabled people use a different Web structure but
make it so they can use it too”**

Dan Fruchterman, Engineer's Focus: Accessible Technology for All

Accessibility helps

- The deaf and hard of hearing
- The blind and partially sighted
- Those with limited dexterity or mobility
- Those with cognitive and/or vestibular disorders

These may be permanent or temporary conditions

Accessibility helps also

- A person holding a phone in one hand
- A person with concussion symptoms
- A person with a broken arm
- A person outside in bright sunlight

These circumstances are also known as **situational disabilities**

Accessibility helps... everyone

Web Accessibility in Law

- In USA – Section 508 (for government institutions, ICT and educational); ADA – non-profits and private sector
- In the European Union – Directive (EU) 2016/2102 of the European Parliament and of the Council of 26 October 2016 on the accessibility of the websites and mobile applications of public sector bodies - WCAG level AA (public sector)
- In Poland – Act on the accessibility of the websites and mobile applications of public sector bodies (19/04/2019) - WCAG level AA (public sector)

WCAG 2.1

Understanding Accessibility Standards



The basics of WCAG

The Web Content Accessibility Guidelines are a series of internationally recognized guidelines that are intended to help solve many problems that web users with disabilities face.

Core principles: Perceivable

Make it easier for users to see and hear content

Think about:

- Alt text
- Captions

Core principles: Operable

Help users navigate to and find content.

Think about:

- Focus states
- Keyboard navigation
- Form labels
- Avoid flashes or fast animations

Core principles: Understandable

Make content appear and operate in predictable ways.

Think about:

- Link text
- Form errors

Core principles: Robust

Maximize compatibility with current and future user tools.

Think about:

- Semantics
- Valid HTML

Conformance levels

All **WCAG guidelines** are assigned one of three levels of conformance (A, AA, AAA) based on a number of factors including.

Level A

- High user impact
- Low impact on presentation/functionality
- Easy to implement

Level AA

- High user impact
- Some impact on presentation/functionality

Level AAA

- Specific user impact
- Some impact on presentation/functionality
- Increased level of difficulty to implement

Assistive technologies



Assistive technologies

Assistive technologies help users with disabilities increase, maintain or improve their functional capabilities:

- Screen readers
- Screen magnifiers
- Eye tracking software
- Keyboard shortcuts
- Braille display

Assistive technologies – try yourself

1. Launch the built-in screen reader, or install it
 - MacOS and iOS launch VoiceOver,
 - Android – launch TalkBack,
 - Windows – install and launch e.g NVDA (open source)
2. Print list of shortcut or touch gesture (mobile) for chosen screen reader.

Assistive technologies – try yourself

- Without a mouse (using only keyboard to navigate)
 - After launching screen reader, open a browser window and enter the address of website (for example pwr.edu.pl)
 - Go through each element on website, hear how these element are reading by screen reader. What kind of information screen reader is giving?
 - Try various shortcut (go through headings, landmarks etc).
 - Open windows with list of links on website and hear them. Do the descriptions of links make sense? Do you know where they provide?
 - Now try to explore website with turned off monitor.

Basic NVDA command

1. Key "NVDA" - Insert/Caps Lock
2. Stop - Ctrl,
3. Close - NVDA + Q
4. Faster/Slower - Ctrl + NVDA + ↑/↓
5. Read previous/next line - ↑/↓
6. Read again current line - NVDA + ↑
7. Read all - NVDA + ↓
8. Form fields/Links/Buttons - Tab
9. Headings – key H, or keys 1-6,
10. Forms - "F"
11. Button - key "B",
12. Links - key "L",
13. Landmarks - key "D",
14. Tables - key "T",
15. Table cell - Ctrl + Alt + ↑/↓/←/→,
16. Navigation backward - Shift (+ Tab, b, etc.)
17. Change form mode - NVDA + Space

Semantic code



Semantics is the base of accessibility

"Elements, attributes, and attribute values in HTML are defined (by this specification) to have certain meanings (semantics). For example, the ol element represents an ordered list, and the lang attribute represents the language of the content".

<https://html.spec.whatwg.org/multipage/dom.html#semantics-2>

Semantic level – formal

1. Formal, described in the specification and intended for users:

- the use of appropriate tags ensures that every user of the Network will understand what we are about to convey,
- the importance of items is directly related to how they behave and how users expect the items to behave.

Semantic level – formal

2. Formal, the infrastructure of which is described in the specifications, and which is intended for browsers and other programs:

- implementation of mechanisms such as microdata, JSON-LD, RDFa,
- providing additional information for "ARIA" assistive technologies,
- expanding semantics.

Semantic level – non-formal

3. Informal, based on the conventions partially described in the specification, and intended for web developers.

Try yourself - Windows and high contrast mode

- Narzędzie usuwa wszystkie style nadane elementom przez developera,
- na podstawie domyślnych arkuszy stylów przeglądarki oraz semantyki poszczególnych elementów wyświetla wszystkie elementy,
- wystylowane elementy oparte na div/ span przestają pełnić swoją funkcję (zostaje tekst),
- użytkownik korzysta z interfejsu zgodnie z swoimi preferencjami.

Semantics - rules

Each element in the code, such as a button, link, header or table, must be marked with a valid HTML element. Thanks to semantics, readers understand:

- element descriptive properties (role, name, value, etc.),
- transient states (pressed, selected, receiving keyboard focus, etc.),
- events (text change, button activation, selection of a check box),
- actions that the user can perform (clicking, selecting, dragging, etc.),
- relationships (parent / child, controls, etc.),
- identify textual content.

Semantic HTML tags

- header
- nav
- article
- section
- aside
- footer
- figure
- figcaption
- details
- summary
- time

Non-semantic HTML tags

- div
- span

WAI-ARIA

Web Accessibility Initiative
- Accessible Rich Internet Applications



What WAI-ARIA it is?

Definition:

Technical specification WAI-ARIA (Web Accessibility Initiative - Accessible Rich Internet Applications) was created with the aim of providing technology that enables improvement the availability of websites and web applications.

1. Adds ARIA attributes to the specified HTML elements, increasing their availability.
2. It is useful for increasing the availability of various custom implementations the type of action, components, including:
 - changing content without having to reload HTML,
 - widgets such as tabs, accordions, etc. (not having HTML equivalents).
 - non-semantic elements (<div>,) (action and role defined in Javascript).

Types of ARIA attributes

Accessible Rich Internet Applications is a collection of attributes added to tags HTML.

- **Roles** - pass information to assistive applications that the element described by it is different from its tag.
- **Properties** - define features or additional meaning of a given element, which are not available in the HTML standard.
- **States** - allow to define the current state of an item.

ARIA - widget roles, describing the purpose of components and their elements

- alert
- dialog
- progressbar
- scrollbar
- slider
- spinbutton
- tab
- tooltip
- tree

More: [https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/ARIA_Techniques#live_region_roles)

[US/docs/Web/Accessibility/ARIA/ARIA_Techniques#live_region_roles](https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/ARIA_Techniques#live_region_roles)

ARIA – role of document structure

- document
- group
- presentation
- toolbar

More: https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/ARIA_Techniques#document_structure_roles

ARIA – landmarks role

Describing the features of each region of the page and providing an additional method of navigation:

application	
banner	<header>
complementary	<aside>
contentinfo	<footer>
form	
main	<main>
navigation	<nav>
search	

ARIA - States and properties

While a specific element can only have one role, it can have a series states and properties that may change in response to user actions. You can them divided into the following groups:

1. Widget attributes
2. Regions attributes
3. Binding attributes

ARIA – Widget attributes

- aria-autocomplete
- aria-checked
- aria-disabled
- aria-expanded
- aria-hidden
- aria-label
- aria-pressed
- aria-readonly
- aria-required
- aria-selected

ARIA – Widget attributes

A set of attributes whose content is updated live (no page refresh)

- aria-atomic
- aria-busy
- aria-live
- aria-relevant

Attributes “drag-and-drop”

- aria-dropeffect (in ARIA 1.1 outdated)
- aria-grabbed (in ARIA 1.1 outdated)

ARIA – Bindings attribute

Describing the relationship between the page elements that cannot be passed through the document structure.

- aria-activedescendant
- aria-controls
- aria-describedby
- aria-flowto
- aria-labelledby
- aria-owns
- aria-posinset
- aria-setsize

Rules of application ARIA

Important!

By using ARIA, we can significantly improve the accessibility of websites for people using screen readers, while using them incorrectly, we can also significantly worsen it.

The W3C has formulated five basic principles for using the ARIA attributes.

ARIA - application rule 1

If you can - use native HTML controls. It is a safer and more universal solution than patching with ARIA

ARIA - application rule 2

Do not change the semantics of a given HTML element, unless absolutely necessary (using ARIA roles).

1. "Roles" can be used to give semantic meaning to non-semantic elements such as `<span>` or `<div>`.
2. We are not changing the role of the HTML semantic element



```
1 <h2 role="button">Nagłówek</h2>
```



```
1 <h2><button>Nagłówek</button></h2>
```

ARIA - application rule 2

Comment:

If for some reason it is not possible to use the `<button>` element, it is better to use an element that has no semantic meaning (e.g. `<span>`) and give it an appropriate "role" attribute:



```
1 <h2><span role="button">Nagłówek</span></h2>
```

ARIA - application rule 3

1. All ARIA interactive elements must be accessible from the keyboard level (JS keyboard events).
2. If the person using the mouse is able to activate, select or drag an item, the person using the keyboard should be able to perform the same actions (not just ARIA).
3. When creating a button using the `<span>`, ARIA and JavaScript elements that the user can click on, we must:
 - make sure that this element will also receive keyboard focus (this can be done by adding the `tabindex = "0"` attribute) and supplementing mouse events with keyboard events in JavaScript),
 - test the correct operation of this element with a screen reader

ARIA - application rule 4

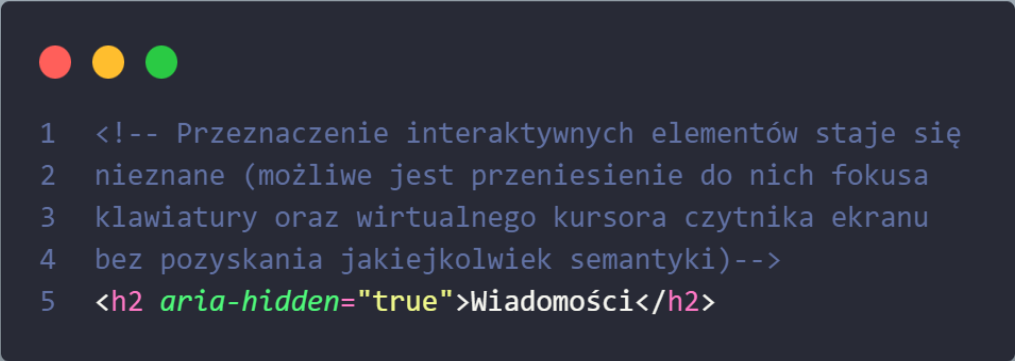
Do not add role = "presentation" or aria-hidden = "true" to visible items that receive keyboard focus.

- The role = "presentation" attribute removes the semantic meaning of the element it is on (leaves the text)



ARIA - application rule 4

- The aria-hidden = "true" attribute removes a given element from the available browser tree (readers ignore it)
- The purpose of the interactive elements becomes unknown (it is possible to transfer the keyboard focus and the virtual screen reader cursor to them without acquiring any semantics).



```
1 <!-- Przeznaczenie interaktywnych elementów staje się  
2 nieznane (możliwe jest przeniesienie do nich fokusa  
3 klawiatury oraz wirtualnego kursora czytnika ekranu  
4 bez pozyskania jakiegokolwiek semantyki)-->  
5 <h2 aria-hidden="true">Wiadomości</h2>
```

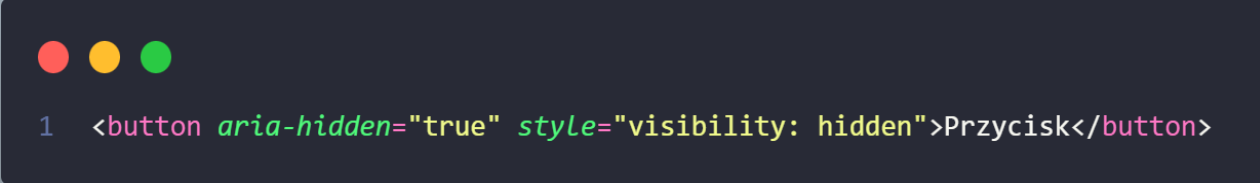
ARIA - application rule 4

- The use of the following solutions is prohibited:



```
1 <button role="presentation">Przycisk</button>
2 <button aria-hidden="true">Przycisk</button>
3 <a aria-hidden="true" href="...">Link</a>
```

- If an interactive element is invisible and inactive then aria-hidden = "true" is allowed, e.g.:



```
1 <button aria-hidden="true" style="visibility: hidden">Przycisk</button>
```

ARIA - application rule 5

All interactive elements must have an accessible name (label).

What does it mean accessible?

One that has been appropriately tagged in the HTML code.

ARIA - application rule 5

- In the case of links / buttons, the "available name" is the text between the tags (<a>, <button>), and (, </button>)

```
1 <a href="[...]">Wiadomości</a>  
2 <button>Znajdź</button>
```

- In the case of links - images, it can also be the content of the "alt" attribute

```
1 <a href="[...]"></a>
```

ARIA - application rule 5

- In the case of form fields, it is the content of the <label> element correctly associated with the field using the "for" and "id" attributes



```
1 <label for="nazwisko">Nazwisko:</label>  
2 <input type="text" id="nazwisko">
```

ARIA - application rule 5

Attention!

The `<span>` or `<p>` element placed before the form field is in no way related to the field that describes

this is not an available name!



that is also not an available name!



```
1  <!-- to nie jest dostępna nazwa! -->
2  <span>Nazwisko:</span>
3  <input type="text" id="nazwisko">
4
5  <!-- to też nie jest dostępna nazwa! -->
6  <input type="text" id="nazwisko" placeholder="Nazwisko">
7
```

ARIA - application rule 5

If it is not possible to use the above-mentioned elements, use the appropriate ARIA attributes to provide a label that will be recognized by screen readers:



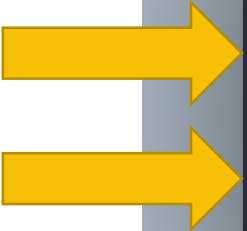
```
1 <input type="text" id="nazwisko" aria-label="nazwisko">
2
3 <span id="nazwisko-label">Nazwisko:</span>
4 <input type="text" id="nazwisko" aria-labelledby="nazwisko-label">
```

ARIA - application rule 6 (extra)

Do not use ARIA where it is not needed!

This is link without aria role too

This is button without aria role too



```
1 <!-- to już jest link! -->
2 <a role="link" href="[...]">Link</a>
3
4 <!-- to i tak jest przycisk! -->
5 <button role="button">Zamknij</a>
```



POLITECHNIKA NOWYCH SZANS



Unia Europejska
Europejski Fundusz Społeczny



Politechnika Wrocławska



Projekt współfinansowany przez Unię Europejską, Europejski Fundusz Społeczny, Program Operacyjny Wiedza Edukacja Rozwój
Tytuł projektu: „Politechnika Nowych Szans”